

AI 데이터 분석 · 11일차

함수

한 번 만들고 여러 번 쓴다

def · 호출 · 이름 · 리팩토링 · 매개변수 · 키워드 · 기본값 · return · 스코프 · 조합

교안 07_01(함수의 기본) · 07_02(매개변수와 반환값) · 07_03(함수 설계와 활용)

여태 써오던 print()가 어떻게 만들어졌는지 알게 되는 날

2026-08-04 AI 데이터 분석 교육 정리 (11일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

지금까지는 **남이 만든 함수를 써왔습니다**. `print()`, `len()`, `input()`, `type()` — 전부 함수입니다. 오늘은 그 함수를 **직접 만드는** 날입니다. 교안 표현으로 "누군가 미리 만들어 둔 함수를 쓰는 단계에서, 내가 직접 만드는 단계로 한 발 나아가는" 시간입니다.

함수 def · 호출 · 이름 **매개변수** 위치 · 키워드 · 기본값 **반환** return · 다중 반환 **스코프** 지역변수 **패턴** 설계 · 조합 · docstring

연결

강사님이 어제 예고하신 두 가지가 오늘 회수됩니다. 첫째, **튜플이 다시 나옵니다** — 함수가 값을 여러 개 돌려줄 때 튜플로 묶어 보내기 때문입니다. 둘째, 미뤘었던 `sorted(key=)`도 함수를 알아야 풀립니다. **오늘이 파이썬 기초의 마지막 관문입니다**.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 왜 함수인가	반복 코드가 부르는 세 문제
STEP 1	함수의 개념과 구조	def의 다섯 부분 · 콜론과 들여쓰기
STEP 2	호출과 실행 흐름	괄호는 실행 버튼 · 점프와 복귀
STEP 3	함수 이름 짓기	문법 규칙 3 · 좋은 이름 3조건
STEP 4	코드 중복과 함수화	찾기→뭉기→교체→확인 · 리팩토링
STEP 5	매개변수 — 값을 건네기	매개변수와 인자 · 개수 규칙
STEP 6	위치 인자와 키워드 인자	순서 사고를 문법으로 막기
STEP 7	return — 결과를 돌려주기	print와의 차이 · 다중 반환 · None
STEP 8	기본값 인자	선택적 매개변수 · 순서 규칙
STEP 9	지역변수와 범위	값의 수명 · 같은 이름 다른 변수
STEP 10	함수 설계와 조합	입력-처리-출력 · docstring · 파이프라인
STEP 11	내 코드 점검 — 수업 코드 3개	실행 흐름 · 남은 TODO
STEP 12	실습 점검 — 기본 3 · 선택 3	제출 현황 · 버그 · 미제출 3문제
부록 A·B·C	치트시트 · 오류 사전 · 예고	한 장 요약 · 함정 모음

오늘의 지도 — 왜 함수인가

복사·붙여넣기를 두 번 이상 하고 있다면

교안 첫 장의 문장이 오늘 전체를 요약합니다 — "복사-붙여넣기를 두 번 이상 하고 있다면, 그건 코드가 잘못 가고 있다는 신호다." 강사님은 여기에 실제 사고 사례를 붙이셨습니다. 20여 년 전 국산 압축 프로그램에서 일정 크기가 넘는 파일을 담으면 앱이 죽는 문제가 크게 터졌는데, 원인은 데이터 처리 코드를 한 줄씩 전부 복사해 붙이다가 **156번을 빠뜨리고 155번을 두 번 쓴 것**이었습니다.

반복 코드가 부르는 세 가지 문제

문제	함수 없이	함수를 쓰면
길이	같은 코드가 화면을 가득 채운다	한 번 정의하고 이름만 부른다
수정	문구 하나 바뀌면 전부 찾아 고쳐야 한다	함수 안 한 줄만 고치면 끝
실수	복사하다 한 줄만 틀려도 사고	틀릴 자리 자체가 사라진다

설비 적용

강사님이 덧붙인 현실적인 이야기입니다 — **회사에서는 그 하루가 곧 비용**입니다. 글자 하나 잘못 쓴 것을 잡아내려고 하루를 쓰면 회사는 그만큼 급여를 지출한 셈이고, 그 가치가 있었는지로 평가받게 됩니다. 배우는 과정에서는 "경험 쌓았다" 하고 넘어갈 수 있지만, 현업에서는 **실수할 여지를 구조적으로 줄여두는 것**이 실력입니다.

함수란 무엇인가

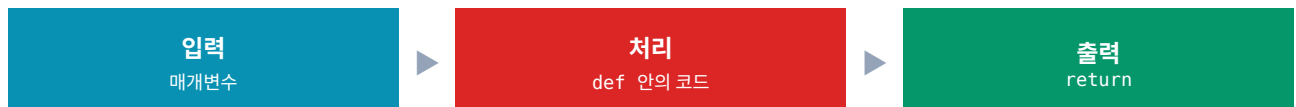
교안의 정의는 한 줄입니다 — "**특정 작업을 하는 코드를 묶고 이름을 붙인 것**." 그리고 이미 우리는 그런 것들을 써왔습니다.

구분	뜻	예
내장 함수	파이썬이 미리 만들어 둔 것 — 바로 쓸 수 있다	<code>print · len · input · type · sorted · sum</code>
사용자 정의 함수	내가 def로 직접 만든 것 — 오늘 배우는 것	<code>check_temp() · calc_average()</code>

비유

교안이 든 비유는 **자판기**입니다 — 동전을 넣으면(입력) 정해진 일을 하고(처리) 음료가 나옵니다(출력). 그리고 자판기를 쓰는 사람은 안에서 무슨 일이 일어나는지 몰라도 됩니다. 강사님도 `print()` 하나를 예로 드셨는데, 문자·숫자·심표 여러 개·f-string 무엇을 넣어도 알아서 처리하는 그 안쪽 코드는 결코 단순하지 않습니다. **우리는 다섯 글자만 쓰면 됩니다.** 함수가 복잡함을 안쪽에 가둬 두기 때문입니다.

오늘 배우는 세 부분이 자판기의 세 부분과 정확히 대응합니다. 지금까지는 가운데(처리)만 있는 함수를 만들 수 있고, 입력은 STEP 5에서, 출력은 STEP 7에서 채워집니다.

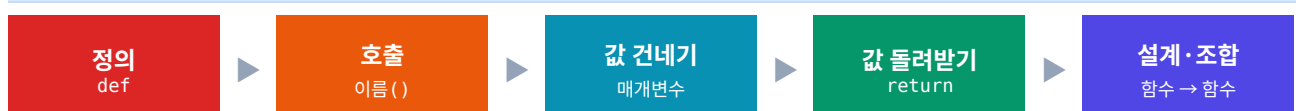


오늘 교안 세 개와 이 책의 대응

오늘 교안은 세 개입니다. 이 책은 세 교안의 흐름을 그대로 따라 STEP을 나눴습니다.

교안	파트	이 책의 STEP
07_01 함수의 기본	개념과 구조 · 호출과 이름 · 코드 중복과 함수화	CH0 · STEP 1~4
07_02 매개변수와 반환값	매개변수의 이해 · 여러 인자와 키워드 인자 · 반환값의 이해	STEP 5~7
07_03 함수 설계와 활용	기본값 인자 · 지역변수와 범위 · 함수 설계와 조합	STEP 8~10

오늘의 흐름



한 문장으로 붙이면 — 일을 이름 붙여 정의하고, 필요할 때 불러서, 값을 건네고, 결과를 돌려받아, 다음 함수로 잇는다. 이것이 오늘 하루의 전부입니다.

STEP 1

함수의 개념과 구조

def의 다섯 부분

교안은 def 한 줄을 다섯 부분으로 쪼개 설명합니다. 기호가 많아 보여도 나눠 보면 단순합니다.

#	부분	예	설명
①	def 키워드	def	"지금부터 함수를 만든다"는 선언
②	함수 이름	check_temp	나중에 부를 때 쓸 이름
③	소괄호	()	받을 값의 자리 — 없으면 빈 괄호
④	콜론	:	"아래가 함수 몸통"이라는 신호
⑤	들여쓰기	4칸	여기부터가 함수 안 — 문법 그 자체

주의

교안이 ④⑤를 따로 떼어 강조한 이유가 있습니다 — 초보자가 함수를 만들 때 가장 많이 내는 에러 두 가지가 콜론 누락과 들여쓰기 실수이기 때문입니다. 조건문·반복문에서 이미 써온 규칙이라 새로운 것은 없지만, 함수는 몸통이 길어지는 경우가 많아 들여쓰기를 놓치기 쉽습니다.

키워드 def 함수를 정의한다

정의 def 이름(): 아래에 **들여쓴 코드**가 그 함수의 몸통이 됩니다. 정의만 해서는 아무 일도 일어나지 않습니다.

형태 def 함수이름(): 실행할 코드

```
def say_hello():
    print("안녕하세요")

# 여기까지는 아무것도 출력되지 않는다

say_hello()      # 호출해야 실행된다
say_hello()
say_hello()
```

▶ 안녕하세요
▶ 안녕하세요
▶ 안녕하세요

왜 쓰나 교안 표현으로 "함수를 만든 것은 레시피를 적어 둔 것일 뿐"입니다. 레시피를 적어 서랍에 넣어 둔 상태이고, 요리를 하려면 따로 꺼내야 합니다. def 줄을 만나면 파이썬은 "이런 이름의 함수가 있다"는 사실만 기억하고 지나갑니다.

응용 정의와 호출이 분리되어 있어서 호출 횟수를 자유롭게 조절할 수 있습니다. 위 코드에서 print("안녕하세요")는 딱 한 번 적혀 있지만 세 번 실행됐습니다. 인사말을 바꾸고 싶으면 함수 안 한 줄만 고치면 세 곳이 모두 바뀝니다.

```
def print_line():
    print("=" * 60)

def print_check():
    print("점검 안내 출력")

for i in range(1, 4):
    print_line()
    print(f"장비 {i}")
    print_check()
```

```
▶ =====
▶ 장비 1
▶ 점검 안내 출력
▶ =====
▶ 장비 2
▶ 점검 안내 출력
▶ =====
▶ 장비 3
▶ 점검 안내 출력
```

자주 하는 실수

콜론을 빠뜨리면 SyntaxError, 들여쓰기를 빠뜨리면 IndentationError입니다. 둘 다 실행 전에 잡히는 오류라 그나마 다행인 편입니다.

문법 들여쓰기 = 함수의 경계 칸으로 코드를 묶는다

정의 파이썬은 중괄호가 아니라 들여쓰기 수준으로 코드 블록을 구분합니다. 들여쓰기가 끝나는 지점이 곧 함수가 끝나는 지점입니다.

```
def check_sensor():
    print("① 함수 안")          # 들여쓰기 안 → 호출할 때만 실행
    print("② 함수 안")

print("③ 함수 밖")            # 들여쓰기 없음 → 바로 실행

check_sensor()
```

- ▶ ③ 함수 밖
- ▶ ① 함수 안
- ▶ ② 함수 안

왜 쓰나 출력 순서가 ③ → ① → ②인 것이 핵심입니다. ③은 함수 밖이라 즉시 실행되고, ①②는 `check_sensor()`를 부른 뒤에야 실행됩니다. **들여쓰기 한 칸 차이가 실행 시점을 완전히 바꿉니다.**

응용 들여쓰기는 보기 좋으라고 하는 게 아니라 **문법 그 자체**입니다. 다른 언어에서 중괄호가 하는 일을 파이썬은 칸으로 합니다. 그래서 실수로 한 줄만 밖으로 빼면 오류 없이 동작만 달라집니다.

```
# 실수 — 마지막 줄을 함수 밖으로 빼먹었다
def check_sensor():
    print("① 점검 시작")
print("② 점검 끝")          # ← 들여쓰기 누락! 함수 밖이 되었다

print("--- 아직 호출 안 함 ---")
check_sensor()
```

- ▶ ② 점검 끝
- ▶ --- 아직 호출 안 함 ---
- ▶ ① 점검 시작

주의

들여쓰기는 **스페이스 4칸**이 파이썬 표준입니다. 탭과 스페이스를 섞으면 `TabError`가 날 수 있으니 편집기 설정을 한쪽으로 통일해 두는 것이 좋습니다.

STEP 2

호출과 실행 흐름

괄호는 실행 버튼

함수 함수 호출 만들어 둔 함수를 실행

정의 함수 이름 뒤에 **소괄호**를 붙이면 실행됩니다. 교안 표현으로 "**괄호는 실행 버튼**"입니다.

형태 함수이름()

```
def greet():  
    print("점검 시작")  
  
greet()  
greet()  
greet()  
  
print(greet)      # 괄호 없음 — 실행되지 않는다
```

```
▶ 점검 시작  
▶ 점검 시작  
▶ 점검 시작  
▶ <function greet at 0x7f8b1c0d2e50>
```

왜 쓰나 괄호가 곧 "**실행하라**"는 신호입니다. 어제 배운 3초 판별법에서 "괄호가 붙으면 함수·메서드"라고 한 이유가 여기 있습니다.

응용 교안이 강조하는 또 하나 — **호출 횟수에 제한이 없고, 매번 깨끗한 상태에서 새로 시작합니다.** 앞 호출에서 무슨 일이 있었든 다음 호출은 처음부터 다시 실행됩니다. 호출끼리 서로 독립적이라는 뜻인데, 이 성질은 STEP 9의 지역변수와 직접 연결됩니다.

```
def show_counter():  
    count = 0          # 호출할 때마다 새로 0  
    count = count + 1  
    print(count)  
  
show_counter()  
show_counter()  
show_counter()  
  
▶ 1  
▶ 1  
▶ 1
```


주의

괄호를 빼먹으면 **오류도 안 나고 실행도 안 됩니다**. 아무 일도 안 일어나는데 오류도 없으면 이 실수를 가장 먼저 의심하세요.

문법 정의 시점과 호출 시점 반드시 만든 다음에 부른다

정의 def가 실행되는 순간에는 **이름만 등록**되고, 실제 코드는 호출할 때 실행됩니다. 그래서 **정의가 호출보다 위에** 있어야 합니다.

```
greet()          # 아직 모르는 이름

def greet():
    print("점검 시작")

# NameError: name 'greet' is not defined
```

왜 쓰나 교안 비유가 정확합니다 — "서랍에 아직 넣지도 않은 레시피를 꺼내려는 것"입니다. 파이썬은 위에서 아래로 읽어 내려가므로, 호출하는 줄에 도달했을 때 그 이름이 아직 등록되지 않았으면 NameError가 납니다.

응용 그래서 파이썬 파일의 표준 구조는 **위쪽에 함수들을 모아 정의하고, 아래쪽에서 호출**하는 형태입니다. 오늘 만든 수업 코드도 자연스럽게 이 모양이 되었습니다.

```
# — 위쪽: 정의 영역 —
def print_line():
    print("=" * 40)

def check(name):
    print(f"{name} 점검")

# — 아래쪽: 실행 영역 —
print_line()
check("압축기A")
print_line()

▶ =====
▶ 압축기A 점검
▶ =====
```

개념

`NameError`는 이름 자체를 모른다는 뜻입니다. 오늘 나올 다른 오류 `UnboundLocalError`(STEP 9)와 헷갈리기 쉬운데, 그쪽은 "이름은 아는데 아직 값이 없다"입니다.

실행 흐름 — 점프와 복귀

교안이 "각주를 보러 잠깐 내려갔다가 다시 본문으로 돌아오는 것"에 비유한 부분입니다. 네 단계로 보면 이렇습니다.



실습 3 (07_01) — 실행 흐름 추적하기

```
def step_one():
    print(" [함수] step_one 시작")
    print(" [함수] step_one 끝")

print("① def는 기억만 — 아직 실행 안 됨")
print("② 호출 직전")
step_one()
print("③ 호출이 끝나고 돌아옴")
step_one()
print("④ 전체 끝")
```

- ▶ ① `def`는 기억만 — 아직 실행 안 됨
- ▶ ② 호출 직전
 - ▶ [함수] `step_one` 시작
 - ▶ [함수] `step_one` 끝
- ▶ ③ 호출이 끝나고 돌아옴
 - ▶ [함수] `step_one` 시작
 - ▶ [함수] `step_one` 끝
- ▶ ④ 전체 끝

개념

교안이 원하는 방법이 좋습니다 — 손가락으로 코드 위를 움직이며 점프와 복귀를 짚어 보기. 위에서 아래로 읽다가 `step_one()`을 만나면 손가락이 위로 올라가 함수 안을 훑고, 끝나면 다시 원래 자리로 내려옵니다. 이 동작이 눈에 그려지면 함수 흐름은 끝난 셈입니다.

STEP 3

함수 이름 짓기

규칙 3가지 + 좋은 이름 3조건

이름은 두 층으로 나눠 봐야 합니다. **지키지 않으면 실행조차 안 되는 문법 규칙**이 있고, 지켜도 실행은 되지만 **읽는 사람이 고생하는 품질 조건**이 있습니다.

① 문법 규칙 — 여기면 SyntaxError

규칙	내용	가능	불가
시작 문자	영문자나 밑줄로 시작 · 숫자로 시작 불가	check_temp	2nd_check
사용 문자	영문자·숫자·밑줄만 · 띄어쓰기 불가	print_data	check temp
예약어 회피	파이썬이 이미 쓰는 단어는 피한다	my_print	print · if · for

x 이름 규칙 위반

```
def 2nd_check(): ... # SyntaxError: invalid decimal literal
def check temp(): ... # SyntaxError: invalid syntax
```

예약어는 오류는 안 나지만 원래 기능이 사라진다

```
def print(x):
    return "oops"
```

```
print("이제 화면에 안 나옴") # 아무것도 출력되지 않는다
```

자주 하는 실수

세 번째가 가장 위험합니다. `print`·`len`·`sum`·`list`·`type` 같은 **내장 함수 이름**을 덮어써도 오류가 나지 않습니다. 그 뒤로 원래 기능을 못 쓰게 될 뿐이죠. 어제 셋을 배울 때 `sorted = sorted(alerts)`로 내장 함수를 덮어쓴 사례와 정확히 같은 계열입니다. **에디터에서 색이 다르게 표시되는 단어는 이름으로 쓰지 않는다고** 기억하면 안전합니다.

② 좋은 이름의 세 조건

조건	뜻	나쁜 예	좋은 예
동작 표현	동사로 시작해 하는 일을 드러낸다	temperature	check_temperature
구체성	무엇을 다루는지까지 담는다	process	process_sensor_data

조건	뜻	나쁜 예	좋은 예
일관성	비슷한 함수는 비슷하게 짓는다	check_temp / pressure_ck	check_temp / check_pressure

앞머리를 관례대로 붙이면 읽는 사람이 더 빨리 이해합니다. 실무에서 널리 쓰이는 접두사입니다.

접두사 관례

```
get_...      # 값을 가져와 돌려준다    get_random_group()
calc_...     # 계산해서 돌려준다     calc_average()
print_...    # 화면에 출력한다 (반환 없음)  print_line()
check_...    # 검사한다               check_sensor()
is_ / has_   # 참·거짓을 돌려준다     is_over_limit()
```

개념

교안 표현으로 **"이름만 잘 지어도 코드 읽는 속도가 크게 달라진다"**입니다. 며칠 뒤 다시 열어 봐도 이름이 명확하면 **함수 안을 들여다보지 않아도** 무슨 함수인지 압니다. 강사님이 "실습 제출할 때 가장 신경 써야 할 것"으로 꼽으신 이유입니다.

③ 한 함수는 한 가지 일

교안이 좋은 함수의 기준으로 든 원칙입니다. 줄 수가 아니라 **하나의 명확한 작업을 담당하는가**가 기준입니다. 한 줄짜리 함수도 좋은 함수일 수 있고, 스무 줄짜리도 좋은 함수일 수 있습니다.

x 이름과 역할이 어긋난 함수

```
def show_student():
    print("학생1: 짱구")
    print("학생2: 철수")
    print("학생3: 훈이")
    print("선생님: 채송화")      # ← 학생을 보여주는 함수인데 선생님이 나온다
```

- ▶ 학생1: 짱구
- ▶ 학생2: 철수
- ▶ 학생3: 훈이
- ▶ 선생님: 채송화

show_student()를 호출한 사람은 학생 명단만 나올 것이라 기대합니다. 이 함수를 **학생 수를 세는 데 썼다면** 결과가 하나 더 나옵니다. 그 사람 코드에는 아무 잘못이 없는데도요.

오류 종류	언제 드러나나	예	찾는 법
문법 오류	실행 전 — 에디터가 빨간 줄	콜론 누락 · 이름 규칙 위반	저절로 보인다
실행 오류	그 줄에 도달할 때 중단	NameError · TypeError	Traceback 맨 아래 줄
논리 오류	드러나지 않음	이름과 역할이 다른 함수	사람이 읽고 대조

주의

강사님 판단 기준이 명확합니다 — **"작동하다가 애한테 더 넣었다 하면, 이름 바꿀 생각부터 해야 한다."** 함수에 기능을 하나 더 붙이려는 순간이 곧 이름을 다시 볼 때이고, 이름을 못 바꾸겠으면 **함수를 나눠야 한다는 신호**입니다. 관련 개념으로 **사이드 이펙트**도 알려주셨는데, 내 코드는 멀쩡한데 다른 부분과 예상치 못한 영향을 주고받는 상황을 가리킵니다.

STEP 4

코드 중복과 함수화

찾기 → 묶기 → 교체 → 확인

함수를 잘 쓰려면 먼저 "여기는 함수로 만들면 좋겠다"는 자리를 알아보는 눈이 필요합니다. 신호는 명확합니다 — 값만 다르고 구조가 똑같은 코드 덩어리.

BEFORE — 값만 다른 같은 구조

```
print("=" * 60)
print("압축기A 점검 안내 출력")
print("=" * 60)
print("펌프B 점검 안내 출력")
print("=" * 60)
print("건조로C 점검 안내 출력")
```

네 단계 절차

STEP	하는 일	위 코드에서는
1. 찾기	반복되는 공통 부분을 식별한다	구분선 + 안내 출력이 3번
2. 묶기	def로 함수를 만들고 이름을 정한다	<code>print_line()</code> · <code>print_check(name)</code>
3. 교체	반복되던 자리를 모두 호출로 바꾼다	반복문으로 3번 호출
4. 확인	전후 출력이 완전히 같은지 대조한다	한 글자라도 다르면 놓친 것

2단계에서 매번 달라지는 값이 곧 매개변수가 됩니다. 위 코드에서는 장비 이름만 다르므로 매개변수는 하나입니다.

AFTER — 함수 + 반복문

```
def print_line():
    print("=" * 60)

def print_check(name):
    # 달라지는 값 = 매개변수
    print(f"{name} 점검 안내 출력")

for eq in ["압축기A", "펌프B", "건조로C"]:
    print_line()
    print_check(eq)
```

```
▶ =====
▶ 압축기A 점검 안내 출력
▶ =====
▶ 펌프B 점검 안내 출력
▶ =====
▶ 건조로C 점검 안내 출력
```

주의

4단계 **확인**을 빠뜨리는 경우가 많습니다. 교안이 따로 한 장을 할애한 이유가 있죠 — 함수로 묶으면서 줄 하나를 빠뜨리거나 순서를 바꿔도 **오류가 안 납니다**. 묶기 전 출력을 파일로 저장해 두고 묶은 뒤 출력과 비교하면 확실합니다. 어제 디셔너리·그제 종합 실습에서 나온 논리 버그들이 전부 이 "확인"을 안 해서 남은 것들이었습니다.

리팩토링 — 동작은 그대로, 구조만 더 좋게

중복 코드를 함수로 묶는 일에는 정식 용어가 있습니다. **리팩토링(refactoring)**입니다. 사용자가 보기에는 프로그램이 똑같이 작동하지만 코드 내부는 더 깔끔해진 상태를 말합니다.

단계	언제	무엇을
① 일단 돌아가게	처음 짤 때	탐색하듯 코드를 짤다 — 중복이 쌓여도 괜찮다
② 작동 확인 후	어느 정도 돌아갈 때	중복을 함수로 정리한다
③ 전후 대조	정리 직후	같은 입력에 같은 결과가 나오는지 확인

설비 적용

교안 표현이 데이터 분석 실무를 정확히 짚습니다 — "**데이터 분석에서는 탐색하듯 코드를 짜다 보면 중복이 쌓이기 쉽다.**" 처음부터 완벽한 함수를 설계하려 하면 오히려 진도가 안 나갑니다. **먼저 동작하게, 그다음 깔끔하게** — 이 순서가 실무의 기본입니다. 다만 "그다음"을 영원히 미루면 압축 프로그램 사고 같은 일이 생기는 것이고요.

STEP 5

매개변수 — 값을 건네기

같은 함수를 다른 상황에

지금까지 만든 함수는 **항상 똑같은 일만** 했습니다. 교안 표현으로 "**함수가 하나의 값에 묶여 있는**" 상태죠. 이름을 바꿔 인사하려면 함수를 새로 만들어야 할까요? 그럴 필요가 없도록 만든 것이 매개변수입니다.

매개변수 매개변수 (parameter) 함수가 받을 값의 자리

정의 def 줄의 소괄호 안에 적는 **변수 이름**. 호출할 때 넘긴 값이 여기에 담깁니다.

형태 def 함수이름(매개변수): 매개변수를 사용하는 코드

```
def say_hi(name):  
    print(f"반갑습니다. {name}")
```

```
say_hi("Ned")  
say_hi("Tuna")  
say_hi("Layla")
```

- ▶ 반갑습니다. Ned
- ▶ 반갑습니다. Tuna
- ▶ 반갑습니다. Layla

왜 쓰나 교안이 든 표현이 좋습니다 — "**빈 괄호에 이름 하나로 함수가 달라진다.**" 괄호 안에 이름 하나 넣었을 뿐인데 함수 하나로 상황마다 다른 결과를 낼 수 있게 됩니다.

응용 설비 점검처럼 **대상만 바꾸고 절차는 같은 작업**에 딱 맞습니다. 그리고 매개변수는 **매번 새로 채워집니다** — 호출할 때마다 그 호출에서 넘긴 값으로 갈아 끼워집니다.

```
def check(name):  
    print(f"{name} 장비의 점검을 시작합니다")  
  
equipment = ["압축기A", "펌프B", "건조로C"]  
for eq in equipment:  
    check(eq)                # eq 값이 매번 name에 새로 담긴다
```

- ▶ 압축기A 장비의 점검을 시작합니다
- ▶ 펌프B 장비의 점검을 시작합니다
- ▶ 건조로C 장비의 점검을 시작합니다

개념

용어를 정확히 구분합니다 — `def check(name)`의 `name`이 매개변수(parameter), `check("압축기A")`의 "압축기A"가 인자(argument)입니다. 교안 표현으로 등장 시점이 다릅니다 — 매개변수는 정의할 때, 인자는 호출할 때 나타납니다. 받는 쪽이 매개변수, 주는 쪽이 인자라고 기억하면 됩니다.

매개변수 매개변수 여러 개 심표로 나열

정의 소괄호 안에 심표로 여러 개를 적습니다. 호출할 때도 같은 개수를 같은 순서로 넘깁니다. 개수에 제한은 없습니다.

형태 `def 함수이름(매개1, 매개2, ...):`

```
def report(name, temp):  
    print(f"{name} {temp} 도")
```

```
report("모터", 78)
```

```
report("펌프", 92)
```

▶ 모터 78 도

▶ 펌프 92 도

왜 쓰나 설비 데이터는 이름·온도·진동처럼 여러 값이 한 세트로 다닙니다. 매개변수를 여러 개 두면 그 세트를 한 번에 넘길 수 있습니다. 위 코드가 **07_02 실습 2**의 정답입니다.

응용 매개변수가 많아지면 **딕셔너리 하나로 묶어 넘기는** 방법도 있습니다. 어제 배운 딕셔너리가 여기서 쓰입니다. 다만 이 방식은 함수 안에서 어떤 키가 필요한지 밖에서 안 보이므로, 키가 고정적일 때만 쓰는 게 좋습니다.

```
def report_sensor(sensor):  
    name = sensor.get("이름")  
    temp = sensor.get("온도")  
    print(f"{name}: {temp}°C")
```

```
report_sensor({"이름": "온도센서A", "온도": 24.5})
```

```
report_sensor({"이름": "온도센서B", "온도": 30.2})
```

▶ 온도센서A: 24.5°C

▶ 온도센서B: 30.2°C

자주 하는 실수

개수가 안 맞으면 즉시 `TypeError`입니다. 교안 표현으로 이건 불편이 아니라 **안전장치**입니다 — 값을 덜 넘긴 채로 조용히 돌아가는 것보다 그 자리에서 멈추는 편이 낫습니다.

규칙 매개변수 이름 정하기 무엇이 들어올지 드러나게

정의 매개변수 이름도 함수 안에서 쓰는 지역 변수입니다. 무엇이 들어올지 알 수 있는 이름을 붙입니다.

```
# 나쁜 이름 — 무엇이 들어오는지 모른다
def report(a, b):
    print(f"{a} {b} 도")

# 좋은 이름 — 호출부를 안 봐도 읽힌다
def report(name, temp):
    print(f"{name} {temp} 도")
```

왜 쓰나 함수 안 코드를 읽을 때 a, b는 매번 위로 올라가 확인해야 하지만 name, temp는 그 자리에서 읽힙니다. 변수 이름은 미래의 나에게 남기는 주석입니다.

응용 짧게 쓰고 싶은 유혹이 있지만, 한 글자 이름은 반복문의 인덱스(i)나 좌표(x, y)처럼 관례가 확립된 경우에만 씁니다. 그 외에는 길더라도 의미가 드러나는 쪽이 낫습니다.

```
# 관례상 허용되는 짧은 이름
for i in range(3):
    ...
def distance(x, y):
    ...

# 그 외에는 의미를 드러낸다
def calc_average(values):
    ...
def judge(avg, limit):
    ...
```

주의

매개변수 이름과 호출할 때 넘기는 변수 이름은 달라도 됩니다. check(eq)로 불러도 함수 안에서는 name으로 받습니다. 값만 전달되고 이름은 각자 쓰는 것입니다.

개수가 안 맞으면 나오는 오류

인자 개수 불일치

```
def check(name, temp):  
    return f"{name} {temp}"
```

```
check("펌프A")
```

```
# TypeError: check() missing 1 required positional argument: 'temp'
```

```
check("펌프A", 37.4, 2.1)
```

```
# TypeError: check() takes 2 positional arguments but 3 were given
```

개념

missing 1 required positional argument: 'temp' — 어느 매개변수가 비었는지 이름까지

알려줍니다. 튜플 언패킹에서 본 not enough values to unpack과 같은 계열의 메시지이고, 원인도 같습니다

— **개수를 맞추라**는 것입니다. 오류 메시지를 끝까지 읽는 습관만 있으면 이 종류는 즉시 해결됩니다.

STEP 6

위치 인자와 키워드 인자

순서 사고를 문법으로 막기

매개변수 위치 인자 순서대로 채워 넣기

정의 값을 적은 순서대로 매개변수에 담습니다. 기본 방식이며, 교안 표현으로 "위치 인자는 순서가 생명"입니다.

```
def report(name, temp):  
    print(f"장비: {name} / 온도: {temp}")
```

```
report("펌프A", 37.4)      # name←"펌프A", temp←37.4
```

```
report(37.4, "펌프A")     # 순서가 바뀌면 그대로 뒤집힌다
```

▶ 장비: 펌프A / 온도: 37.4

▶ 장비: 37.4 / 온도: 펌프A

왜 쓰나 짧고 간결합니다. `print("안녕")`처럼 매개변수가 하나뿐이거나 순서가 명백할 때는 이 방식이 자연스럽습니다.

자주 하는 실수

두 번째 출력이 문제입니다. **오류가 나지 않습니다.** 장비 이름 자리에 온도가, 온도 자리에 이름이 들어갔는데 프로그램은 태연히 돌아갑니다. 매개변수가 3개, 4개로 늘어나면 이 실수가 급격히 잦아집니다. **07_02 실습 2의 ④ 단계**가 이걸 직접 확인해 보라는 문제입니다.

매개변수 키워드 인자 이름을 지정해 넘기기

정의 매개변수이름=값 형태로 넘깁니다. 순서와 무관하게 정확히 그 자리에 들어갑니다.

형태 함수이름 (매개변수이름=값, ...)

```
def report(name, temp):  
    print(f"{name} {temp}")
```

```
report(temp=78, name="모터")  # 순서를 바꿔도
```

```
report(name="펌프", temp=92)  # 정확히 들어간다
```

▶ 모터 78

▶ 펌프 92

왜 쓰나 강사님이 짚으신 지점이 중요합니다 — "C언어나 자바는 인자를 어떻게 보냈는지 호출부만 봐서는 알 수 없는데, 파이썬은 명시할 수 있게 만들었다." 순서가 바뀌어 생기는 오류를 문법 차원에서 차단한 것입니다. 위 코드가 07_02 실습 3의 정답입니다.

응용 교안이 든 키워드 인자의 두 장점은 ① 순서 자유 ② 의미 명확입니다. `calc(3, 7, 1)`은 세 숫자가 뭔지 알 수 없지만 `calc(width=3, height=7, depth=1)`은 즉시 읽힙니다.

```
def set_alarm(sensor, high, low, unit):
    print(f"{sensor}: {low}~{high}{unit}")

# 위치 인자 — 숫자만 봐서는 뭐가 뭔지 모른다
set_alarm("모터온도", 90, 20, "C")

# 키워드 인자 — 호출부만 봐도 읽힌다
set_alarm(sensor="모터온도", low=20, high=90, unit="C")
```

▶ 모터온도: 20~90C
▶ 모터온도: 20~90C

자주 하는 실수

섞어 쓸 때는 위치 인자가 먼저 와야 합니다. `report(name="A", 37.4)`처럼 쓰면 `SyntaxError: positional argument follows keyword argument`가 납니다. 07_02 실습 3의 ④단계가 이 규칙을 확인하는 문제입니다.

두 방식 비교

	위치 인자	키워드 인자
형태	<code>report("펌프A", 37.4)</code>	<code>report(temp=37.4, name="펌프A")</code>
순서	반드시 정의 순서대로	바뀌도 된다
길이	짧다	길어진다
가독성	매개변수가 많으면 알기 어렵다	호출부만 봐도 읽힌다
섞을 때	반드시 앞에	반드시 뒤에

주의

강사님도 모든 호출에 키워드 인자를 강제하지는 않는다고 하셨습니다. `print("안녕")` 처럼 매개변수가 명백한 경우까지 이름을 붙일 필요는 없죠. 기준은 매개변수가 많아질수록, 그리고 같은 자료형이 나란히 있을수록 키워드 인자입니다. 숫자 세 개를 나란히 넘기는 함수라면 거의 반드시 키워드 인자가 낫습니다. 강사님 권장은 "직접 만든 함수를 호출할 때만큼은 키워드 인자를 쓰기로 약속해 두는 것"입니다.

이미 써오던 키워드 인자

사실 키워드 인자는 오늘 처음 보는 게 아닙니다. 내장 함수를 쓰면서 이미 여러 번 썼습니다. 그때는 문법 이름을 몰랐을 뿐이죠.

배운 날	코드	키워드 인자 부분
1일차	<code>print("a", "b", sep=" ")</code>	<code>sep</code> — 구분자 지정
1일차	<code>print("진행중", end="")</code>	<code>end</code> — 줄바꿈 대신 쓸 문자
9일차	<code>sorted(temps, reverse=True)</code>	<code>reverse</code> — 내림차순 여부
9일차	<code>enumerate(sensors, start=1)</code>	<code>start</code> — 시작 번호
10일차	<code>d.get("유량", 0)</code>	(이건 위치 인자 — 기본값은 함수 정의 쪽)

공통점이 보입니다 — 전부 생략 가능한 옵션이고, 생략하면 기본값이 쓰입니다. `print`는 `sep=" "`, `sorted`는 `reverse=False`, `enumerate`는 `start=0`이 기본값이죠. 즉 이 내장 함수들은 기본값 인자(STEP 8)로 정의되어 있고, 우리는 그걸 키워드로 덮어써 온 것입니다.

집작해 보는 내장 함수 정의

```
# 내장 함수의 정의를 집작해 보면 이런 모양일 것이다
def print(*values, sep=" ", end="\n"): ...
def sorted(iterable, reverse=False): ...
```

연결

앞으로 라이브러리 문서를 읽을 때 이 감각이 그대로 쓰입니다. `pd.read_csv(path, encoding="utf-8", header=0)`을 보면 — `path`는 필수 위치 인자, `encoding`과 `header`는 기본값이 있는 선택 인자라는 것이 바로 읽힙니다. 강사님이 "원리를 알아야 활용법도 제대로 이해된다"고 하신 것이 이 얘기입니다.

STEP 7

return — 결과를 돌려주기

print는 사람에게, return은 프로그램에게

교안이 한 줄로 정리한 문장이 이 STEP의 전부입니다 — **"print는 사람에게, return은 프로그램에게."** 화면에 보여주는 것과 값을 손에 쥐는 것은 전혀 다른 일입니다.

	print	return
결과	화면에 글자로 보인다	값으로 돌아온다
받는 대상	사람의 눈	프로그램(변수·다음 함수)
변수에 담으면	None	그 값
다음 계산에	쓸 수 없다	바로 쓸 수 있다
함수 이름 예	print_report()	calc_average() · get_sensor()

키워드 return 값을 돌려주고 함수를 끝낸다

정의 return 값으로 결과를 호출한 쪽에 돌려줍니다. 교안 표현으로 **"return은 두 가지를 동시에"** 합니다 — 값 반환과 함수 종료.

형태 return 값

```
def add(a, b):  
    return a + b  
  
v = add(40.0, 45.0)      # 반환값을 변수에 담기  
print(v)  
print(add(v, 5.0))      # 담은 값을 다음 계산에 이어쓰기  
  
▶ 85.0  
▶ 90.0
```

왜 쓰나 위 코드가 **07_02 실습 4**의 정답입니다. 요구사항 ③④가 "호출 결과를 변수에 담고, 담은 값을 다음 계산에 이어쓰기"였습니다. return이 있기에 가능한 일입니다.

응용 값을 받는 방법은 두 가지입니다 — **변수에 담거나, 곧바로 다른 곳에 넣거나**. 교안은 입문 단계에서 **변수에 담는** 쪽을 권합니다. 중간값에 이름이 붙어 무슨 일이 일어나는지 분명해지기 때문입니다.

```
def get_average(values):
    return sum(values) / len(values)

# (1) 변수를 거쳐 — 중간값이 보여 읽기 쉬움 (권장)
avg = get_average([75, 88, 91])
print(round(avg, 2))

# (2) 바로 넘기기 — 짧지만 여러 겹이면 읽기 어려움
print(round(get_average([75, 88, 91]), 2))
```

```
▶ 84.67
▶ 84.67
```

개념

return은 **예약어(키워드)**입니다. 메서드가 아니므로 점을 찍지 않고 괄호도 필요 없습니다. 어제 배운 3초 판별법으로 보면 "점 없음 + 괄호 없음 = 키워드"입니다.

반환 return의 함수 종료 효과 그 아래 코드는 실행되지 않는다

정의 return을 만나면 함수 실행이 **그 자리에서 끝납니다**. 아래에 코드가 남아 있어도 실행되지 않습니다.

```
def add(a, b):
    return a + b
    print("이 줄은 절대 실행되지 않는다")    # 에디터가 흐리게 표시

print(add(1, 2))
```

```
▶ 3
```

왜 쓰나 강사님이 "**리턴 만나는 건 거의 끝**"이라고 표현하셨습니다. 함수 안 지역 변수들이 사라지기 직전에 값을 밖으로 던지는 마지막 동작인 셈입니다.

응용 이 성질을 역이용하면 **조건에 따라 일찍 끝내는** 코드를 쓸 수 있습니다. 조건이 여러 갈래일 때 return을 여러 개 두면, 어느 갈래로 가든 그 자리에서 끝나므로 else가 필요 없어집니다.


```
def grade_temp(temp):
    if temp > 90:
        return "위험"          # 여기서 끝
    if temp >= 80:
        return "주의"          # 또는 여기서 끝
    return "정상"              # 둘 다 아니면 여기

for t in [95, 85, 70]:
    print(t, "→", grade_temp(t))
```

- ▶ 95 → 위험
- ▶ 85 → 주의
- ▶ 70 → 정상

주의

return이 여러 개 있어도 **실제로 실행되는 것은 하나**뿐입니다. 처음 만나는 return에서 함수가 끝나기 때문입니다.

반환 여러 값 return 침표로 나열하면 튜플이 된다

정의 return 값1, 값2처럼 침표로 나열하면 파이썬이 **자동으로 튜플로 묶어** 돌려줍니다.

형태 return 값1, 값2, ...

```
def calc_statistics(values):
    """목록을 받아 (최솟값, 최댓값, 평균)을 돌려준다."""
    mean = sum(values) / len(values)
    return min(values), max(values), mean

result = calc_statistics([78, 92, 85])
print(result, type(result).__name__)

low, high, mean = calc_statistics([78, 92, 85]) # 언패킹
print(low, high, mean)
```

- ▶ (78, 92, 85.0) tuple
- ▶ 78 92 85.0

왜 쓰나 8일차에 배운 "**괄호 없이 침표만 있어도 튜플**"이 그대로 적용됩니다. 강사님이 어제 예고하신 "**함수를 배울 때 튜플이 왜 필요한지 알게 된다**"가 바로 이 자리입니다. 위 코드가 **07_02 실습 5**의 정답입니다.

응용 받는 쪽에서 **튜플 언패킹**으로 바로 나눠 담으면 인덱스를 쓸 필요가 없습니다. 9일차에 배운 `name, value = sensor`가 여기서 재등장합니다. 딕셔너리에서 값을 골라 튜플로 돌려주는 것도 자주 쓰는 형태입니다.

```
import random

def get_random_group():
    """무작위 그룹의 (이름, 리더)를 돌려준다."""
    groups = [
        {"이름": "에스파", "리더": "카리나"},
        {"이름": "리센느", "리더": "원희"},
    ]
    g = random.choice(groups)
    return g.get("이름"), g.get("리더")

name, leader = get_random_group()
print(f"{name}의 리더는 {leader}입니다")
```

▶ 에스파의 리더는 카리나입니다 (실행할 때마다 달라짐)

자주 하는 실수

언패킹 규칙은 그대로입니다 — **return하는 값의 개수와 받는 변수의 개수가 같아야** 합니다. 다른면 `ValueError: too many values to unpack`이 납니다.

반환 `return`이 없으면 `None` `None`이 알려주는 흔한 실수

정의 `return`이 없는 함수는 자동으로 `None`을 돌려줍니다.

```
def calc_average(a, b):
    print(f"평균 온도: {(a + b) / 2}")    # 화면에 찍기만 한다

avg = calc_average(75.3, 88.0)
print(avg)
```

▶ 평균 온도: 81.65
▶ None

왜 쓰나 수업 코드에 그대로 나온 사례입니다. 화면에는 평균이 잘 찍혔는데 `avg`에는 `None`이 담깁니다. 눈에 보이는 것과 손에 쥔 것이 다른 상황이죠. 교안이 이걸 "**None이 알려주는 흔한 실수**"라고 이름 붙인 이유입니다.

응용 `None`을 그냥 두면 다음 계산에서 터집니다. `avg + 10`을 하는 순간 `TypeError`가 나죠. 그래서 "**이 함수는 화면에 보여주는 함수인가, 값을 돌려주는 함수인가**"를 만들 때부터 정하고 이름에 담아야 합니다.

```
# 보여주는 함수 — print 계열, return 없음
def print_average(a, b):
    print(f"평균 온도: {(a + b) / 2}")

# 돌려주는 함수 — calc/get 계열, return 있음
def calc_average(a, b):
    return (a + b) / 2

print_average(75.3, 88.0)          # 화면 출력용
avg = calc_average(75.3, 88.0)    # 값을 받아 다음 계산에
print("반올림:", round(avg, 1))

▶ 평균 온도: 81.65
▶ 반올림: 81.7
```

주의

결과가 **None**이면 **return**을 빠뜨렸는지 먼저 확인하세요. 교안의 "함수 작성 시 흔한 실수" 표에도 **return** 누락
→ 결과가 **None**으로 올라 있습니다.

STEP 8

기본값 인자

거의 항상 같은 값을 매번 적지 않기

매개변수 기본값 인자 생략하면 쓰이는 값

정의 def 줄에서 매개변수=기본값으로 적어 두면, 호출할 때 생략 가능해집니다.

형태 def 함수이름(필수매개, 선택매개=기본값):

```
def report(name, temp, unit="C"):
    print(f"{name} {temp}{unit}")

report("펌프A", 75.3)           # unit 생략 → "C"
report("펌프A", 167.5, "F")    # 위치로 덮어쓰기
report("펌프A", 167.5, unit="F") # 키워드로 덮어쓰기
```

```
▶ 펌프A 75.3C
▶ 펌프A 167.5F
▶ 펌프A 167.5F
```

왜 쓰나 교안이 짝은 기본값의 두 가치는 ① 편의 ② 유연성입니다. 대부분의 경우 값이 정해져 있는데 매번 적기 번거롭고, 그러면서도 필요할 때는 바꿀 수 있어야 하죠. 온도 단위는 거의 항상 섭씨이므로 기본값으로 두고, 화씨일 때만 지정합니다.

응용 단위를 데이터에 명시하는 것은 실무의 암묵적 규칙입니다. 강사님이 **NASA 사고**를 예로 드셨는데, 미터법과 야드파운드법을 섞어 쓰다 우주선을 잃은 사건입니다. 야구 중계에서 우리는 150km/h를, 미국은 95mph를 말하는 것과 같은 차이죠. **계측 데이터를 다룰 때 단위를 함께 담아 두는 습관**이 이런 사고를 막습니다.

```
def to_celsius(value, unit="C"):
    """온도와 단위를 받아 섭씨로 바꿔 돌려준다."""
    if unit == "F":
        return round((value - 32) * 5 / 9, 1)
    return value

print(to_celsius(75.3))           # 이미 섭씨
print(to_celsius(167.5, "F"))    # 화씨 → 섭씨
```

```
▶ 75.3
▶ 75.3
```

자주 하는 실수

기본값이 있는 매개변수는 반드시 뒤쪽에 와야 합니다. `def report(unit="C", name, temp)` 처럼 쓰면 `SyntaxError: non-default argument follows default argument`가 납니다. 교안 표현으로 "필수가 앞, 선택이 뒤"입니다. 앞쪽을 생각하면 뒤쪽 값이 어디로 들어갈지 정할 수 없기 때문입니다.

필수 매개변수와 선택적 매개변수

구분	정의 형태	호출할 때	위치
필수 매개변수	<code>name</code>	반드시 넘겨야 한다	앞쪽
선택적 매개변수	<code>unit="C"</code>	생략 가능	뒤쪽

07_03 실습 1 — 기본값 인자 함수 만들기

```
def judge(avg, limit=90.0):  
    """평균과 기준값을 받아 상태를 돌려준다. 기준 생략 시 90."""  
    if avg > limit:  
        return "위험"  
    return "정상"  
  
print(judge(85.0))           # 기본 기준 90  
print(judge(85.0, 80.0))    # 위치로 덮어쓰기  
print(judge(85.0, limit=80.0)) # 키워드로 덮어쓰기
```

- ▶ 정상
- ▶ 위험
- ▶ 위험

설비 적용

기본값을 무엇으로 둘지는 어제 딕셔너리에서 배운 판단과 같은 문제입니다. **임계값 기본값을 0으로 두면 모든 센서가 위험으로 잡히고, 무한대로 두면 아무도 안 잡힙니다.** 여기서는 "대부분의 설비에 적용되는 표준 기준"을 기본값으로 두고, 예외 설비만 호출할 때 덮어쓰는 것이 자연스럽습니다.

STEP 9

지역변수와 범위

불편이 아니라 안전장치

함수 안에서 만든 변수는 **그 함수 안에서만** 존재하고, 함수가 끝나면 사라집니다. 이 범위를 **스코프(scope)**라고 합니다. 처음에는 불편해 보이지만 교안이 못 박은 대로 **불편이 아니라 안전장치**입니다.

스코프 지역변수 함수 안에서만 사는 변수

정의 함수 안에서 만들어진 변수. 함수가 끝나면 **메모리에서 사라져** 바깥에서는 볼 수 없습니다.

```
def show_counter():
    count = 0          # 이 함수 안에서만 존재
    count = count + 1
    print(count)

show_counter()
show_counter()
show_counter()

# print(count)
# NameError: name 'count' is not defined
```

```
▶ 1
▶ 1
▶ 1
```

왜 쓰나 출력력이 1, 2, 3이 아니라 **1, 1, 1**인 것이 핵심입니다. 호출할 때마다 `count`가 새로 0부터 만들어지고 함수가 끝나면 사라지기 때문입니다. 교안 표현으로 **"지역변수의 값의 수명"**이 함수 실행 시간과 같습니다.

응용 이 수명이 **독립성**을 만듭니다. 함수를 여러 개 만들어도 각자 안에서 `count`, `total`, `i` 같은 흔한 이름을 마음껏 써도 서로 충돌하지 않습니다. 교안이 **"같은 이름 다른 변수"**라고 부른 성질이죠. 사이드 이펙트가 구조적으로 차단됩니다.

```
def count_sensors():
    total = 3          # 이 total 과
    return total

def count_alerts():
    total = 7          # 이 total 은 완전히 별개
    return total

total = 100            # 바깥의 total 도 또 별개
print(count_sensors(), count_alerts(), total)

▶ 3 7 100
```

개념

매개변수도 지역변수입니다. `def check(name)`의 `name`은 함수 안에서만 살고 끝나면 사라집니다. 교안이 따로 한 장을 쓴 이유는, 매개변수를 특별한 무언가로 오해하기 쉽기 때문입니다. **함수 안에 있는 변수는 전부 지역변수**라고 보면 됩니다.

스코프 `UnboundLocalError` 함수 안에서 대입하면 바깥 변수가 안 보인다

정의 함수 안에서 어떤 변수에 **값을 대입하는 줄이 하나라도 있으면** 파이썬은 그 변수를 지역변수로 취급합니다. 그러면 대입 전에 읽을 수 없습니다.

```
my_number = 24          # 바깥 변수

def show_number():
    print(my_number)     # ← 여기서 오류
    my_number = 44       # 이 줄 때문에 my_number가 지역변수가 된다

show_number()
# UnboundLocalError: local variable 'my_number' referenced before assignment
```

왜 쓰나 수업에서 강사님이 주석 처리해 두신 코드가 정확히 이 상황입니다. 바깥에 `my_number = 24`가 분명히 있는데도 오류가 납니다. **아래 줄의 대입 하나 때문에 함수 전체에서 지역변수로 결정되기** 때문입니다.

응용 `count = count + 1`도 같은 오류입니다. 오른쪽 `count`를 읽으려는데 그 `count`는 아직 만들어지지 않은 지역변수입니다. 수업 코드에서 이 줄을 주석 처리하고 `count = 0`을 먼저 넣은 이유가 그것입니다.

```
# x 오류
def bad():
    count = count + 1    # 오른쪽 count가 아직 없다

# ✓ 먼저 만들어 둔다
def good():
    count = 0
    count = count + 1
    return count

print(good())
```

▶ 1

자주 하는 실수

`print(my_number)`만 있고 대입이 없으면 **바깥 변수를 읽을 수는 있습니다**. 문제는 "읽기 + 대입"을 섞을 때 생깁니다. 교안이 원하는 방식은 애초에 섞지 않는 것 — **함수는 깔끔한 통로로만 소통한다**입니다.

함수는 깔끔한 통로로만 소통한다

교안이 한 장을 할애한 원칙입니다. 함수가 바깥과 값을 주고받는 **정상적인 통로는 두 개**뿐입니다.

방향	통로	권장하지 않는 방법
바깥 → 함수	매개변수로 건넨다	전역 변수를 함수 안에서 직접 읽기
함수 → 바깥	return으로 돌려준다	함수 안에서 전역 변수를 고치기

개념

오른쪽 방식이 왜 나쁜가 하면, **함수가 바깥 상태에 의존하게 되기** 때문입니다. 그러면 그 함수는 혼자서는 동작하지 않고, 바깥 어딘가가 바뀌면 조용히 결과가 달라집니다. STEP 3에서 말한 사이드 이펙트죠. 반대로 **"인자로 받아 return으로 돌려주는"** 함수는 바깥과 무관하게 **항상 같은 입력에 같은 결과**를 냅니다. 테스트하기도, 재사용하기도 쉬운 형태입니다.

실습 2 — 안과 밖이 별개임을 확인

07_03 실습 2 — 지역변수 관찰하기

```
def make_report():  
    title = "설비 점검 보고서"      # 지역변수  
    count = 3  
    print(f"[함수 안] {title} / {count}건")  
    return count                    # 밖으로 내보내려면 return  
  
title = "바깥 제목"                # 같은 이름이지만 완전히 별개  
result = make_report()
```

```
print(f"[함수 밖] title = {title}") # 바깥 것 그대로  
print(f"[함수 밖] result = {result}") # return으로 받은 값  
# print(count) → NameError: 함수 안 count는 이미 사라졌다
```

-
- ▶ [함수 안] 설비 점검 보고서 / 3건
 - ▶ [함수 밖] title = 바깥 제목
 - ▶ [함수 밖] result = 3

STEP 10

함수 설계와 조합

작은 함수를 엮어 큰 작업으로

여기부터는 문법이 아니라 **설계**입니다. 함수를 만들 줄 아는 것과 잘 만드는 것은 다르고, 교안 마지막 파트가 그 차이를 다룹니다.

① 입력-처리-출력 사고법

교안이 제시한 절차는 단순합니다 — **막연히 코드부터 짜지 말고, 무엇을 입력받고 · 어떻게 처리하고 · 무엇을 출력할지 먼저 정한다.**

함수	입력	처리	출력
평균 구하기	숫자 목록	합을 개수로 나눔	평균값
온도 판정	온도, 기준	비교	정상 / 위험
최댓값 찾기	숫자 목록	탐색	최댓값

개념

입력은 매개변수로, 처리는 함수 안 코드로, 출력은 return으로 옮기면 함수의 뼈대가 완성됩니다. 교안이 원하는 방법은 세 가지를 **종이에 먼저 적어 보는 것**입니다. 오늘 CHAPTER 0에서 본 자판기 모델이 그대로 설계 도구가 되는 셈이죠.

② 함수 하나에 기능 하나

판별법이 명확합니다 — **"이 함수가 하는 일을 한 문장으로 말할 수 있는가?"**

신호	예	판정
한 문장으로 깔끔히 설명됨	"센서값의 평균을 구한다"	좋은 함수
"그리고"가 여러 번 나옴	"평균을 구하고, 비교하고, 출력하고, 저장하고..."	나뉘야 한다
너무 잘게 쪼갬	한 줄짜리 함수가 수십 개	이것도 좋지 않다

③ 함수를 조합하는 두 방식

방식	설명	예
순서대로 호출	함수를 차례로 부르며 결과를 넘긴다	정리 → 계산 → 출력
안에서 호출	큰 함수가 작은 함수들을 불러 쓴다	보고서 함수 안에서 평균·판정 호출

07_03 실습 3 — 처리 흐름 만들기

```
def get_average(values):
    """숫자 목록을 받아 평균을 돌려준다."""
    return sum(values) / len(values)

def judge(avg, limit=90.0):
    """평균과 기준값을 받아 상태를 돌려준다."""
    if avg > limit:
        return "위험"
    return "정상"

# (1) 변수를 거쳐 넘기기 — 중간값이 보여 읽기 쉬움 (입문 단계 권장)
avg = get_average([78, 92, 85])
status = judge(avg)
print(f"평균 {avg} → {status}")

# (2) 변수 없이 바로 넘기기 — 짧지만 여러 겹이면 읽기 어려움
print(judge(get_average([78, 92, 85])))
```

▶ 평균 85.0 → 정상

▶ 정상

주의

교안이 **입문 단계에서는 (1) 변수를 거치는 방법**을 권합니다. 중간 결과에 이름이 붙으니 무슨 일이 일어나는지 분명하고, 그 중간값을 다른 곳에도 쓸 수 있기 때문입니다. (2)는 짧지만 함수가 두 겹 세 겹 겹치면 **안쪽부터 거꾸로 읽어야** 해서 금세 어려워집니다. 어제 종합 실습에서 `sorted(...)[ :3]` 순서를 헷갈렸던 것과 비슷한 종류의 부담입니다.

④ docstring — 함수에 설명 달기

문법 docstring 함수 사용 설명서

정의 함수를 정의한 **바로 다음 줄**에 따옴표 세 개(""")로 설명문을 적습니다. 무엇을 받고 무엇을 돌려주는지를 적습니다.

형태 def 함수이름(매개변수): """설명문""" 코드

```
def get_average(values):
    """숫자 목록을 받아 평균을 돌려준다."""
    return sum(values) / len(values)

print(get_average.__doc__)    # docstring 꺼내 보기
```

▶ 숫자 목록을 받아 평균을 돌려준다.

왜 쓰나 교안이 든 세 가지 이점 — ① 쓰는 사람에게 사용법을 알려주고 ② 몇 주 뒤의 나에게 기억을 되살려 주고 ③ 협업할 때 사용 설명서가 됩니다. 주석(#)과 달리 파이썬이 값으로 보관하므로 나중에 꺼내 볼 수도 있습니다.

응용 짧으면 한 줄, 길면 여러 줄로 씁니다. 여러 줄로 쓸 때는 무엇을 하는지 → 무엇을 받는지 → 무엇을 돌려주는지 순서가 일반적입니다. VS Code에서 함수 이름에 마우스를 올리면 이 내용이 툴팁으로 뜹니다.

```
def calc_statistics(values):
    """센서값 목록의 통계를 구한다.

    values: 숫자 목록 (비어 있으면 안 됨)
    반환: (최솟값, 최댓값, 평균) 튜플
    """
    mean = sum(values) / len(values)
    return min(values), max(values), mean
```

주의

이름만으로 명확한 짧은 함수는 생략해도 됩니다. print_line()에 굳이 설명을 달 필요는 없죠. 복잡한 함수, 특히 매개변수가 여러이거나 반환값이 튜플인 함수에는 다는 습관이 좋습니다.

⑤ 함수 설계 체크리스트

교안이 마지막에 제시한 네 가지 질문입니다. 함수를 만든 뒤 스스로에게 물어봅니다.

#	점검 항목	질문
01	기능	한 문장으로 설명되는가
02	이름	이름만 봐도 아는가
03	입출력	입력과 출력이 분명한가
04	동작	여러 입력으로 테스트했는가

자주 하는 실수

04가 오늘 우리에게 가장 필요한 항목입니다. 교안이 든 예가 정확합니다 — "평균 함수가 양수에는 잘 동작해도 빈 목록에서는 문제가 생길 수 있다." 실제로 `get_average([])`는 `ZeroDivisionError`가 납니다. 어제 종합 실습에서 빈 리스트로 평균을 내다 터진 것과 같은 자리죠. 여러 입력으로 테스트는 정상값만이 아니라 0·빈 값·음수·문자를 넣어 보는 것을 뜻합니다.

⑥ 데이터 처리 파이프라인 맛보기

교안이 마지막 장에서 보여준 큰 그림입니다. 각 단계가 독립된 함수라서 한 단계를 점검하거나 바꿀 때 다른 단계에 영향을 주지 않습니다.



07_03 실습 4 — 센서 분석 함수 세트

```
def remove_outliers(values, limit=200.0):
    """기준을 넘는 이상값을 걸러 낸다."""
    clean = []
    for v in values:
        if v <= limit:
            clean.append(v)
    return clean

def get_average(values):
    """숫자 목록의 평균을 돌려준다."""
    return sum(values) / len(values)

def format_result(avg):
    """평균을 보기 좋은 문장으로 만든다."""
    return f"평균 온도: {round(avg, 1)}°C"

raw = [78, 92, 85, 9999, 88]          # 9999는 센서 오류값
clean = remove_outliers(raw)         # STEP 2
avg = get_average(clean)             # STEP 3
print(format_result(avg))            # STEP 4
```

▶ 평균 온도: 85.8°C

연결

교안 마지막 문장이 오늘의 결론입니다 — "우리가 배운 함수가 바로 이 파이프라인을 이루는 부품." 앞으로 pandas를 배우면 이 단계들이 이미 만들어진 함수로 제공됩니다. `df.dropna()`(거르기) → `df.mean()`(통계) → `df.to_csv()`(정리) 같은 식이죠. 구조는 똑같고, 부품을 직접 꺾느냐 가져다 쓰느냐만 다릅니다. 지금 직접 만들어 보는 이유가 여기 있습니다.

오늘의 설계 원칙 다섯 줄

교안 마지막 파트를 압축하면 다섯 문장입니다. 함수를 만들 때마다 이 순서로 짚으면 됩니다.

#	원칙	실전에서
1	입력·처리·출력을 먼저 정한다	코드 치기 전에 세 줄로 적어 본다
2	한 함수는 한 가지 일만	"그리고"가 두 번 나오면 나눈다
3	이름만 봐도 알게 짓는다	동사 + 대상 (<code>calc_average</code>)
4	복잡하면 <code>docstring</code> 을 단다	무엇을 받고 무엇을 돌려주는지
5	여러 입력으로 테스트한다	정상값 · 0 · 빈 값 · 음수 · 문자

5번을 실제로 해보면 오늘 만든 `get_average()`에도 구멍이 있습니다. 교안이 든 예 그대로입니다.

여러 입력으로 테스트

```
def get_average(values):  
    """숫자 목록을 받아 평균을 돌려준다."""  
    return sum(values) / len(values)  
  
print(get_average([78, 92, 85]))    # 정상값 → OK  
print(get_average([]))              # 빈 목록 → ZeroDivisionError  
  
▶ 85.0  
▶ ZeroDivisionError: division by zero
```

방어를 더한 버전 — 기본값 인자까지 활용

```
def get_average(values, default=0.0):  
    """숫자 목록의 평균을 돌려준다. 목록이 비면 default를 돌려준다."""  
    if len(values) == 0:  
        return default  
    return sum(values) / len(values)  
  
print(get_average([78, 92, 85]))  
print(get_average([]))  
print(get_average([], default=-1))    # "데이터 없음"을 음수로 구분
```

```
▶ 85.0  
▶ 0.0  
▶ -1
```

개념

고친 버전에는 오늘 배운 것이 세 개나 들어 있습니다 — **기본값 인자**(`default=0.0`), **early return**(빈 목록이면 즉시 반환), **docstring**(동작과 예외 상황을 한 줄로 설명). 그리고 어제 배운 판단도 재등장합니다 — 결측을 0으로 둘지 -1로 둘지는 "**0이 실제 측정값일 수 있는가**"로 결정합니다.

STEP 11

내 코드 점검 — 수업 코드 3개

실행 흐름과 남은 TODO

오늘 만든 수업 파일은 17_function1.py, 17_function2.py, 17_function_local_7.py 세 개입니다. 대체로 정확하고, 강사님이 일부러 보여주신 "틀린 예"까지 주석으로 잘 남겨 두셨습니다. 몇 군데만 짚습니다.

17_function1.py — 스코프 실험이 잘 기록됨

주석 처리된 `show_number()` 코드가 오늘의 핵심 함정을 그대로 담고 있습니다. 주석에 적어두신 문장도 정확합니다 — **"show_number 안의 my_number와 바깥 my_number는 별개의 존재"**.

주석으로 남긴 스코프 실험

```
# my_number = 123
# def show_number():
#     print(f"함수 시작: {my_number}")
#     my_number = 44
# my_number = 24
# show_number()
```

```
# → UnboundLocalError: local variable 'my_number' referenced before assignment
```

개념

한 가지만 정확히 해두면 좋겠습니다. 주석에 **"그런 게 없다고 뜬다"**고 적으셨는데, 실제 오류는 `NameError(없다)`가 아니라 `UnboundLocalError(있긴 한데 아직 값이 없다)`입니다. 미묘하지만 의미가 다릅니다 — 파이썬은 `my_number`를 지역변수로 인식은 했고, 다만 대입이 아래에 있어 아직 값이 없다고 말하는 것입니다. 이 구분을 알면 "왜 바깥 변수를 못 보지?"가 아니라 "아, 아래 대입 때문에 지역변수가 됐구나"로 바로 이어집니다.

show_student() — 일부러 만든 나쁜 예에 오타 하나

주석에 **"잘못 만든 함수가 되는거임"**이라고 적어두신 그대로입니다. 다만 출력 라벨에 오타가 있습니다.

라벨 오타

```
def show_student():
    print("학생1: 짱구")
    print("학생1: 철수")      # ← 학생2
    print("학생1: 훈이")     # ← 학생3
    print("선생님: 채송화")   # ← 이름과 역할 불일치 (의도된 나쁜 예)
```

- ▶ 학생1: 짱구
- ▶ 학생1: 철수
- ▶ 학생1: 훈이
- ▶ 선생님: 채송화

자주 하는 실수

세 명이 전부 "학생1"로 나옵니다. **복사·붙여넣기 후 숫자를 안 고친 것인데**, 공교롭게도 오늘 CHAPTER 0에서 배운 그 사고 유형과 정확히 같습니다 — 압축 프로그램에서 155번을 두 번 쓴 것과 구조가 같죠. 오류도 안 나고 실행도 됩니다. **작은 코드에서도 복붙 실수는 이렇게 생깁니다.** STEP 4의 4단계 "확인"이 필요한 이유입니다.

17_function2.py — 반환값 실험

calc_average를 만들고 avg에 담아 출력하신 부분이 오늘의 핵심 실험입니다. 실행하면 **평균은 화면에 나오는데 avg는 None입니다.**

수업 코드 그대로

```
def calc_average(a, b):
    print(f"평균 온도: {(a + b) / 2}")  # print만 있고 return이 없다

avg = calc_average(75.3, 88.0)
print(avg)
```

- ▶ 평균 온도: 81.65
- ▶ None

목적	고치는 방법	결과
값을 받아 다음 계산에 쓰고 싶다	<code>return (a + b) / 2</code>	<code>avg = 81.65</code>
화면에만 보이면 된다	그대로 두고 avg에 담지 않는다	오해가 사라진다
둘 다 필요하다	return 하고 호출부에서 print	<code>print(calc_average(...))</code>

키워드 인자 실험도 정확합니다. `report_keywords(temp=37.4, name="펌프A")`로 순서를 바꿔 호출하고, 주석에 "순서가 바뀌었을 때 생겨나는 오류를 근본적으로 차단시켰다"고 적으신 것이 교안의 설명과 정확히 일치합니다.

17_function_local_7.py — 파일 정리

`get_random_group()`이 `17_function2.py`와 거의 같은 내용으로 두 파일에 들어 있습니다. 연습 과정이라 자연스러운 일이지만, 나중에 다시 볼 때 어느 쪽이 최종인지 헷갈립니다. 오늘 배운 docstring을 쓰면 파일 안에서도 성격이 드러납니다.

```
docstring을 붙인 버전
import random

def get_random_group():
    """무작위 그룹의 (이름, 리더)를 돌려준다. 17_function2.py 예제의 확장판."""
    groups = [
        {"이름": "에스파", "리더": "카리나"},
        {"이름": "엔믹스", "리더": "해원"},
        {"이름": "리센느", "리더": "원희"},
    ]
    my_group = random.choice(groups)
    return my_group.get("이름"), my_group.get("리더")

group_name, group_leader = get_random_group()
print(f"{group_name}의 리더는 {group_leader}입니다")
```

▶ 리센느의 리더는 원희입니다 (실행할 때마다 달라짐)

개념

두 파일 모두 `groups/group`이라는 같은 이름을 쓰는데, 함수 밖에도 `groups` 리스트가 있고 함수 안에도 `groups`가 있습니다. 오늘 배운 스코프 덕분에 서로 간섭하지 않아 문제는 없습니다 — 교안이 말한 "같은 이름 다른 변수"의 실제 사례죠. 다만 읽는 사람 입장에서는 헷갈리므로 바꿀 것은 `group_names`처럼 다르게 두는 편이 낫습니다. 동작이 되는 것과 읽기 좋은 것은 별개입니다.

세 파일을 한 흐름으로 읽기

파일	다룬 것	대응 교안
<code>17_function1.py</code>	def · 호출 · 스코프 · 이름과 역할	07_01 전체 + 07_03 PART 2
<code>17_function2.py</code>	매개변수 · 키워드 인자 · return · 튜플 반환	07_02 전체
<code>17_function_local_7.p</code>	<code>random.choice</code> + 딕셔너리 + 다중 반환	07_02 PART 3 응용

파일	다른 것	대응 교안
y		

오늘 배운 것 전부 — 한 화면

```
import random
```

```
def get_random_sensor():                # ① 정의 · 매개변수 없음
    """무작위 센서의 (이름, 값)을 돌려준다.""" # ② docstring
    sensors = [                          # ③ 지역변수 (함수 밖에서는 안 보임)
        {"이름": "모터온도", "값": 92.4},
        {"이름": "펌프압력", "값": 2.1},
    ]
    s = random.choice(sensors)
    return s["이름"], s["값"]            # ④ 다중 반환 → 튜플

def grade(value, limit=80.0):            # ⑤ 기본값 인자
    """값과 기준을 받아 상태를 돌려준다."""
    if value > limit:
        return "위험"                   # ⑥ early return
    return "정상"

name, value = get_random_sensor()        # ⑦ 튜플 언패킹
print(f"{name} {value} → {grade(value)}") # ⑧ 함수 조합
print(f"{name} {value} → {grade(value, limit=50.0)}") # ⑨ 키워드 인자
```

▶ 모터온도 92.4 → 위험

▶ 모터온도 92.4 → 위험

개념

①~⑨가 오늘의 목차와 그대로 대응합니다. **25줄 안에 하루치가 다 들어갑니다.** 이 코드를 처음부터 직접 쳐 보면서 각 줄이 몇 번 항목인지 짚어보는 것이, 오늘 복습으로는 가장 효율이 좋습니다. 특히 ⑦ 튜플 언패킹은 9일차 내용이 재등장한 것이라 두 회차를 한 번에 복습하는 효과가 있습니다.

STEP 12

실습 점검 — 기본 3 · 선택 3

제출 현황과 미제출 문제

오늘 제출 과제는 별도 PDF(20260804 실습문제)로 나왔습니다. **기본문제 3개 중 2개 이상 제출**이 요건이고, 선택문제 3개는 "제출 없어도 괜찮지만 하나라도 더 풀면 도움이 된다"고 안내되어 있습니다.

구분	문제	내용	제출
기본	07_02 실습 2	다중 매개변수로 센서값 계산하기	제출 · 방식 다름
기본	07_02 실습 3	키워드 인자로 함수 호출하기	미제출
기본	07_02 실습 4	반환값으로 간단 계산기 만들기	제출 · 정확
선택	07_02 실습 5	센서 통계 함수 만들기	제출 · 버그 3건
선택	07_03 실습 3	처리 흐름 만들기	미제출
선택	07_03 실습 4	센서 분석 함수 세트 만들기	미제출

개념

기본 요건(3개 중 2개)은 충족했습니다 — 실습 2와 4를 냈고 선택문제까지 하나 풀었으니까요. 다만 **실습 3(키워드 인자)**이 빠졌는데, 이건 오늘 강사님이 특히 강조하신 개념이라 짧더라도 풀어 두는 편이 좋겠습니다. 다섯 줄이면 끝납니다. 그리고 07_03 실습 3·4는 오늘 마지막 파트(함수 설계와 조합)를 확인하는 문제라 이걸 풀어야 하루가 마무리됩니다.

실습 2 — 요구사항과 다른 방식으로 풀었습니다

문제의 예상 결과는 모터 78 도 / 펌프 92 도이고, 요구사항은 **매개변수 두 개를 심표로 정의**하는 것이었습니다. 제출 코드는 **딕셔너리 두 개를 받는 방식**이라 요구사항보다 어렵게 푸셨는데, 정작 ④단계(인자 순서를 바꾸면 결과가 어떻게 달라지는지 확인)는 빠졌습니다.

제출 코드 — 실습 2

```
def report_sensor_val(sensor1, sensor2):
    name1 = sensor1.get("이름")
    temp1 = sensor1.get("온도")
    name2 = sensor2.get("이름")
    temp2 = sensor2.get("온도")
    return (name1, temp1), (name2, temp2)    # 값을 꺼내기만 하고 계산이 없다
```

▶ (('온도센서A', 24.5), ('온도센서B', 30.2))

요구사항대로 — 예상 결과와 일치

```
def report(name, temp):
    """장비 이름과 온도를 받아 한 줄로 출력한다."""
    print(f"{name} {temp} 도")

report("모터", 78)          # ③ 순서대로 전달
report("펌프", 92)

print("--- 순서를 바꾸면 ---")
report(78, "모터")         # ④ 오류 없이 뒤집힌 채로 출력된다
```

▶ 모터 78 도
▶ 펌프 92 도
▶ --- 순서를 바꾸면 ---
▶ 78 모터 도

주의

④단계가 이 문제의 핵심입니다. **순서를 바꿔도 오류가 나지 않고 뒤집힌 채로 출력되는 것**을 직접 봐야, 다음 문제 (실습 3 키워드 인자)가 왜 필요한지 납득이 됩니다. 두 문제가 한 세트로 설계된 것이죠. 덕셔너리로 푸신 방식도 유효한 풀이지만, **요구사항의 학습 의도를 건너뛰게 됩니다.**

실습 3 — 미제출 (기본문제)

요구사항 네 단계가 그대로 코드가 됩니다. 예상 결과는 `모터 78 (순서무관) / 펌프 92`입니다.

실습 3 완성본

```
def report(name, temp):                # ① 매개변수 두 개
    """장비 이름과 온도를 출력한다."""
    print(f"{name} {temp}")

report(temp=78, name="모터")           # ② 이름을 지정해 전달
report(name="펌프", temp=92)           # ③ 순서를 바꿔도 같은 결과

# ④ 섞어 쓸 때는 위치 인자가 먼저
report("건조로", temp=101)             # OK
# report(name="건조로", 101)          # SyntaxError
```

- ▶ 모터 78
- ▶ 펌프 92
- ▶ 건조로 101

실습 4 — 정확합니다

calc_oper()의 결과를 다음 호출의 인자로 넘긴 부분이 오늘 배운 **함수 조합**의 정확한 예입니다. return이 있기에 가능한 코드죠.

제출 코드 — 실습 4 (정확함)

```
def calc_oper(a, b):
    return a + b

num = calc_oper(10, 20)    # 30
print(num)

num2 = calc_oper(20, num)  # 20 + 30 = 50 - 앞 결과를 인자로
print(num2)
```

- ▶ 30
- ▶ 50

응용

한 걸음 더 나가면 함수 이름을 calc_oper(연산) 대신 add(덧셈)로 두는 편이 정확합니다. 지금 함수는 덧셈만 하는데 이름은 "연산"이라 범위가 넓거든요. STEP 3의 **구체성** 조건에 걸리는 부분입니다. 나중에 뺄셈이 필요해지면 calc_oper에 기능을 추가하고 싶어지는데, 그 순간이 **"이름 바꿀 생각부터 해야 하는 때"**입니다.

실습 5 — 버그 3건이 겹쳐 있습니다

센서 통계 함수 자체(`calc_statistics`)는 정확합니다. 세 값을 튜플로 돌려주는 형태도 오늘 배운 그대로입니다. 문제는 **입력을 받는 while 루프** 쪽입니다.

제출 코드 — 실습 5

```
ls = []
while True:
    v = input("센서 값 입력(q입력 시 입력 종료): ")
    if v == "q":
        break
    elif float(v):                # ← ① 0 이면 거짓 · ② 숫자가 아니면 여기서 크래시
        ls.append(float(v))
    else:
        ("입력값 오류")          # ← ③ print가 없어 아무것도 안 나온다
        continue

print(calc_statistics(ls))       # ← ④ 값이 하나도 없으면 크래시
```

버그	입력	실제 결과
① 0을 조용히 버림	24.5 / 0 / 30.2 / q	답긴 값 [24.5, 30.2] — 0이 사라짐
② 숫자 아닌 값에 크래시	24.5 / abc / q	ValueError: could not convert string to float
③ 오류 안내가 안 보임	(else에 도달 시)	아무것도 출력되지 않음
④ 빈 입력 후 통계	q	ZeroDivisionError: division by zero

개념

①이 가장 미묘합니다. `elif float(v):`는 "**변환한 값이 참이면**"이라는 뜻입니다. 파이썬에서 `0.0`은 거짓이므로 0을 입력하면 `else`로 빠집니다. 센서값 0은 **실제로 측정될 수 있는 값**인데 통계에서 빠지는 것이죠 — 어제 덕셔너리에서 본 "기본값 0의 함정"과 완전히 같은 계열입니다. ②는 더 심각한데, 애초에 `float("abc")`에서 터지므로 **else 블록에는 영원히 도달하지 못합니다**. 오류 처리를 하려고 만든 코드가 실행조차 되지 않는 구조입니다. ④는 오늘 배운 **설계 체크리스트 04번(여러 입력으로 테스트했는가)**에 정확히 걸리는 항목입니다.

완성본 — 네 가지 경우 모두 검증

```
def calc_statistics(values):
    """센서값 목록을 받아 (최솟값, 최댓값, 평균)을 돌려준다."""
    mean = sum(values) / len(values)
    return min(values), max(values), mean
```

```

values = []
while True:
    raw = input("센서 값 입력(q 입력 시 종료): ")

    if raw == "q":
        break
    if raw == "":
        continue # 빈 입력은 무시

    # 숫자인지 먼저 확인 — 변환 전에 걸려야 크래시가 안 난다
    if not raw.replace(".", "", 1).replace("-", "", 1).isdigit():
        print(" ⚠ 숫자가 아닙니다. 다시 입력하세요.")
        continue

    values.append(float(raw)) # 0도 정상 측정값이므로 그대로 담는다

if len(values) == 0:
    print("입력된 값이 없습니다.")
else:
    low, high, mean = calc_statistics(values) # 튜플 언패킹
    print(f"입력 {len(values)}건 · 최소 {low} · 최대 {high} · 평균 {round(mean, 2)}")

```

▶ [24.5 / 0 / 30.2 / q 입력 시]
▶ 입력 3건 · 최소 0.0 · 최대 30.2 · 평균 18.23
▶
▶ [24.5 / abc / 30.2 / q 입력 시]
▶ ⚠ 숫자가 아닙니다. 다시 입력하세요.
▶ 입력 2건 · 최소 24.5 · 최대 30.2 · 평균 27.35
▶
▶ [q 만 입력 시]
▶ 입력된 값이 없습니다.

주의

("입력값 오류")는 **소괄호로 감싼 문자열일 뿐**입니다. 파이썬은 이 줄을 보고 "문자열을 하나 만들었구나" 하고 그냥 버립니다. 오류도 안 나고 화면에도 안 나옵니다. `print("입력값 오류")`로 써야 출력됩니다. 함수 호출은 **이름 + 괄호**인데 여기서 이름이 빠진 셈이죠. STEP 2에서 "괄호를 빼먹으면 아무 일도 안 일어난다"고 한 것과 짝을 이루는 실수입니다.

07_03 실습 3 · 4 — 미제출 (선택문제)

두 문제 모두 STEP 10에 완성본을 실어 두었습니다. 요구사항만 다시 정리하면 이렇습니다.

문제	요구사항	예상 결과
실습 3	① 계산해 return하는 함수 ② 판정해 출력하는 함수 ③ 첫 결과를 변수에 담아 둘째에 전달	평균 85.0 → 정상
실습 4	① 목록→평균 return ② 평균+기준(기본값)→상태 return ③ 두 함수 연결	85.0 정상

실습 3·4 완성본 — 예상 결과와 일치

```
def get_average(values):
    """숫자 목록을 받아 평균을 돌려준다."""
    return sum(values) / len(values)

def judge(avg, limit=90.0):
    """평균과 기준값을 받아 상태를 돌려준다. 기준 생략 시 90."""
    if avg > limit:
        return "위험"
    return "정상"

temps = [78, 92, 85]
avg = get_average(temps)          # ③ 결과를 변수에 담아
status = judge(avg)               # 다음 함수에 전달
print(f"평균 {avg} → {status}")
print(f"{avg} {judge(avg)}")      # 실습 4 예상 결과 형식
```

▶ 평균 85.0 → 정상

▶ 85.0 정상

치트시트

함수 한 장 요약

정의와 호출

	문법	설명
키워드	<code>def 이름():</code>	함수 정의 — 이 줄만으로는 실행 안 됨
호출	<code>이름()</code>	실행. 괄호가 곧 "실행 버튼"
문법	<code>: + 4칸 들여쓰기</code>	콜론과 들여쓰기가 함수의 경계
매개변수	<code>def 이름(a, b):</code>	받을 값의 자리 (parameter) — 정의할 때
인자	<code>이름(5, 8)</code>	실제로 넘기는 값 (argument) — 호출할 때
키워드 인자	<code>이름(b=8, a=5)</code>	순서 무관 · 위치 인자보다 뒤에
기본값	<code>def 이름(a, unit="C"):</code>	생략 가능 · 정의에서 뒤쪽에 배치
키워드	<code>return 값</code>	값 반환 + 함수 즉시 종료
반환	<code>return a, b</code>	튜플로 묶여 나감 → 언패킹으로 받음
문법	<code>"""설명문"""</code>	docstring — def 바로 다음 줄

꼭 기억할 다섯 가지

#	내용	확인 방법
1	정의만으로는 실행되지 않는다 — 호출해야 돈다	괄호가 있는지 본다
2	정의가 호출보다 위에 있어야 한다	NameError면 순서 확인
3	함수 안 변수는 함수가 끝나면 사라진다	밖에서 쓰면 NameError
4	return이 없으면 반환값은 None	변수에 담아 print 해본다
5	return을 만나면 아래 코드는 실행되지 않는다	에디터가 흐리게 표시

함수 설계 4단계

단계	질문	도구
① 입력	무엇을 받을 것인가	매개변수
② 처리	무엇을 할 것인가 — 한 문장으로 되는가	def 안의 코드
③ 출력	무엇을 돌려줄 것인가	return
④ 점검	이름은 명확한가 · 여러 입력으로 테스트했는가	docstring · 테스트

자주 쓰는 패턴 4가지

외워둘 네 덩어리

① 값을 받아 계산해 돌려준다 (가장 기본)

```
def calc_average(values):  
    """숫자 목록의 평균을 돌려준다."""  
    return sum(values) / len(values)
```

② 여러 값을 돌려주고 언패킹으로 받는다

```
def calc_statistics(values):  
    return min(values), max(values), sum(values) / len(values)  
low, high, mean = calc_statistics([78, 92, 85])
```

③ 조건에 따라 일찍 끝낸다 (early return)

```
def grade(temp, limit=90.0):  
    if temp > limit:  
        return "위험"  
    return "정상"
```

④ 함수를 이어 붙인다 (변수를 거쳐 — 입문 권장)

```
avg = calc_average([78, 92, 85])  
status = grade(avg)  
print(f"평균 {avg} → {status}")
```

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 교안이 꼽은 흔한 실수 4가지

실수	증상	확인할 점
괄호 누락	실행 안 됨 (오류도 없음)	호출에 괄호가 있는지
return 누락	결과가 None	return이 있는지
인자 수 불일치	TypeError	매개변수 개수 맞추기
정의 전 호출	NameError	정의를 위로 올리기

B-2 · 오늘 실제로 겪은 것

x 이렇게 하면	무슨 일이	✓ 바로잡기
<code>elif float(v):</code>	0을 입력하면 거짓이라 조용히 버려짐	변환 전에 숫자인지 먼저 검사
<code>float("abc")</code>	ValueError로 즉시 중단 — else에 도달 못 함	<code>.isdigit()</code> 등으로 사전 검사
<code>("입력값 오류")</code>	아무것도 출력되지 않음 (그냥 문자열)	<code>print("입력값 오류")</code>
빈 리스트로 평균 계산	ZeroDivisionError	<code>if len(values) == 0:</code> 먼저 방어
print만 하고 return 없음	변수에 담으면 None	값이 필요하면 return을 둔다
복붙 후 숫자 안 고침	"학생1"이 세 번 — 오류 없음	매개변수 + 반복문으로 대체
같은 이름 함수 재정의	앞의 정의를 조용히 덮어씀	실습마다 파일을 나눈다

B-3 · 함수 문법 함정

x 이렇게 하면	무슨 일이	✓ 바로잡기
<code>say_hello</code>	아무 일도 안 일어남 (오류도 없음)	<code>say_hello()</code>
<code>def f()</code>	SyntaxError — 콜론 누락	<code>def f():</code>
def 다음 줄 들여쓰기 없음	IndentationError	4칸 들여쓴다

x 이렇게 하면	무슨 일이	✓ 바로잡기
<code>def 2nd_check():</code>	SyntaxError — 숫자로 시작	<code>def check_2nd():</code>
<code>def check temp():</code>	SyntaxError — 띄어쓰기	<code>def check_temp():</code>
<code>def print(x):</code>	내장 함수를 덮어쓰 (오류 없음)	내장 이름은 피한다
<code>check(name="A", 37.4)</code>	SyntaxError — 위치 인자가 키워드 뒤	위치 인자를 앞으로
<code>def f(unit="C", name):</code>	SyntaxError — 기본값 인자가 앞	<code>def f(name, unit="C"):</code>
<code>count = count + 1</code>	UnboundLocalError (초기화 없이)	함수 안에서 먼저 <code>count = 0</code>
<code>return</code> 아래에 코드	절대 실행되지 않음 (오류 없음)	<code>return</code> 위치를 확인

B-4 · 오류 메시지 읽는 법

메시지	뜻	볼 곳
<code>missing 1 required positional argument</code>	인자를 덜 넘겼다 (이름까지 알려줌)	호출부
<code>takes 2 positional arguments but 3 were given</code>	인자를 더 넘겼다	호출부
<code>name 'x' is not defined</code>	그 이름이 아예 없다 — 정의 순서 확인	정의 위치
<code>local variable referenced before assignment</code>	함수 안 대입 때문에 지역변수가 됨	함수 안
<code>positional argument follows keyword argument</code>	호출 시 인자 순서 규칙 위반	호출부
<code>non-default argument follows default argument</code>	정의 시 매개변수 순서 규칙 위반	def 줄

개념

교안의 결론이 좋습니다 — **"실수 목록을 외우기보다 여러 메시지를 읽는 습관을 들인다."** 오늘 나온 오류는 대부분 메시지가 원인을 정확히 말해 줍니다. 문제는 B-2의 절반처럼 **오류 없이 결과만 틀리는 것들**입니다 — 0이 버려지고, 아무것도 출력되지 않고, 함수가 조용히 덮어써지는 경우죠. 이걸 메시지가 안 나오므로 **예상 출력을 먼저 적어두고 대조**하는 수밖에 없습니다. 교안이 STEP 4의 "확인" 단계와 설계 체크리스트 04번을 강조한 이유입니다.

실습 하이라이트 & 다음 시간 예고

기초 문법의 마지막 관문

C-1 · 오늘 자료와 작성 파일

파일	내용	상태
Lecture/07_01_함수의_기본	개념과 구조 · 호출과 이름 · 코드 중복과 함수화 (52p)	전 범위 진도
Lecture/07_02_매개변수와_반환값	매개변수 · 여러 인자와 키워드 · 반환값 (50p)	전 범위 진도
Lecture/07_03_함수_설계와_활용	기본값 인자 · 지역변수와 범위 · 설계와 조합 (51p)	전 범위 진도
Python/17_function1.py	def · 호출 · 스코프 · 이름과 역할	정확 · 라벨 오타 1건
Python/17_function2.py	매개변수 · 키워드 인자 · return · 튜플 반환	정확 (None 실험 포함)
Python/17_function_local_7.py	random.choice + 딕셔너리 + 다중 반환	정확 · 중복 예제
Practice/07_01_example.py	07_01 실습 1~4	2건 완료 · 2건 미완성
Practice/07_02_example.py	실습 2·4·5	기본 요건 충족 · 버그 3건

C-2 · 오늘 배운 것 한눈에

분류	오늘 나온 것	개수
키워드	def · return	2개
개념	정의와 호출 · 실행 흐름 · 스코프 · 지역변수 · 사이드 이펙트 · 리팩토링	6개
인자 방식	위치 인자 · 키워드 인자 · 기본값 인자	3개
반환	단일 반환 · 다중 반환(튜플) · None · early return	4개
설계	입력-처리-출력 · 한 기능 원칙 · 조합 2방식 · docstring · 체크리스트	5개
내장 함수	random.choice · abs · min · max · sum	5개

C-3 · 파이썬 기초 완주 지도

오늘로 파이썬 기초 문법이 사실상 끝났습니다. 지금까지 걸어온 길을 한눈에 보면 이렇습니다.

구간	배운 날	내용	한 줄 요약
입출력·연산	1~4일차	print · 변수 · 자료형 · 연산자 · 입력	값을 다룬다
문자열	4~5일차	인덱싱 · 슬라이싱 · 메서드 · f-string	텍스트를 다룬다
제어 흐름	7~8일차	if · for · while · break · continue	흐름을 다룬다
자료구조	6~10일차	리스트 · 튜플 · 셋 · 딕셔너리	데이터를 담는다
함수	11일차	def · 매개변수 · return · 설계	코드를 묶는다
데이터 분석	다음	모듈 · 파일 입출력 · pandas	실제 데이터로

C-4 · 다음 시간 예고

다음 주제	한 줄 미리보기
모듈과 경로 다루기	import random처럼 남이 만든 함수 묶음을 가져다 쓰기. 교안 08_01
파일 입출력	CSV를 읽어 split으로 나누고 딕셔너리 리스트로. 교안 08_02 — 10일차 실습 7의 본편
sorted의 key=	9일차에 미뤄둔 숙제. 정렬 기준을 함수로 넘기는 문법이라 오늘 배운 것이 전제였다
pandas	강사님 표현으로 "사실상 함수 덩어리". 오늘 배운 호출·인자·반환이 그대로 쓰인다

연결

강사님이 오늘 여러 번 말씀하신 관점이 있습니다 — "데이터 분석을 하다 보면 함수를 직접 만들 일이 없는 경우도 생긴다." 이미 만들어진 함수를 조합해 쓰는 일이 훨씬 많기 때문입니다. 그럼 왜 만드는 법을 배우느냐 — 원리를 알아야 활용법도 제대로 이해되기 때문입니다. `pd.read_csv(path, encoding="utf-8")`을 볼 때 "아, 첫 번째는 위치 인자고 두 번째는 키워드 인자에 기본값이 있겠구나"가 바로 읽히는 것과, 그냥 외워서 쓰는 것은 다릅니다.

오늘의 성취와 다음 한 걸음

쓰는 사람에서 만드는 사람으로

오늘 배운 것을 한 문장으로 줄이면 — 반복되는 일에 이름을 붙여 정의하고, 필요한 값을 건네고, 결과를 돌려받는다. 첫날부터 써온 `print()`가 어떤 원리로 만들어졌는지 이제 알게 되었습니다.

그리고 오늘로 파이썬 기초 문법이 사실상 끝났습니다. 값·문자열·흐름·자료구조·함수 — 앞으로 마주칠 코드는 이 다섯 가지의 조합입니다. 새로 배울 것은 문법이 아니라 라이브러리, 즉 남이 만들어 둔 함수 묶음을 쓰는 법입니다.

오늘 헛갈렸던 지점 다시 짚기

헛갈린 것	정리
매개변수 vs 인자	<code>def f(name)</code> 의 name 이 매개변수(정의할 때), <code>f("펌프A")</code> 의 "펌프A"가 인자(호출할 때)
왜 <code>count</code> 가 계속 1인가	호출할 때마다 지역변수가 새로 만들어지고 함수가 끝나면 사라진다
왜 <code>avg</code> 가 <code>None</code> 인가	<code>print</code> 는 사람에게 보여줄 뿐 값을 돌려주지 않는다. 값이 필요하면 <code>return</code>
<code>return</code> 아래 코드	실행되지 않는다. <code>return</code> 은 값 반환과 함수 종료를 동시에 한다
키워드 인자를 왜 쓰나	순서가 바뀌어도 정확히 들어간다. 순서 사고를 문법으로 차단하는 장치
기본값 인자 위치	필수가 앞, 선택이 뒤. 앞에 두면 <code>SyntaxError</code>
<code>NameError</code> vs <code>UnboundLocalError</code>	전자는 이름 자체를 모른다(정의 순서), 후자는 이름은 아는데 아직 값이 없다(지역변수)

개념

복습 루틴 추천: ① `def`로 함수 하나 만들고 정의만 하고 호출 안 했을 때 아무 일도 안 일어나는 것을 눈으로 확인 → ② `count = 0; count += 1; print(count)` 함수를 3번 호출해 1, 1, 1이 나오는 것을 확인 → ③ `print`만 있는 함수를 변수에 담아 `None`이 나오는 것을 확인 → ④ 미제출인 07_02 실습 3(키워드 인자)을 STEP 12를 덮고 직접 풀기 — 다섯 줄이면 끝납니다 → ⑤ 07_03 실습 3·4를 풀어 함수 조합까지 마무리. ①~③은 각각 3줄이면 확인되고, 직접 짚어봐야 몸에 남습니다. ④⑤가 오늘 남은 숙제입니다.